Personal Study Notes

Mona Kumari

Syngene / BBRC Digital Pathology Assessment

Nucleus Detection & Quantification

Self-Study Companion

May 2026

Contents

Chapter 1

00 — Overview (the 3-minute summary)

1.1 The task in one sentence

Detect every nucleus in 50 H&E histopathology images covering 5 cancer types (10 each), report counts and morphometrics, and present results. **No ground-truth annotations were provided.**

1.2 The dataset

Cohort	Disease
BRC	Breast carcinoma
CRC	Colorectal carcinoma
HCC	Hepatocellular carcinoma
NSCLC_AD	Non-small-cell lung carcinoma — adenocarcinoma
NSCLC_SCC	Non-small-cell lung carcinoma — squamous cell carcinoma

Each PNG is roughly 480 × 560 px, RGB. Nuclei are blue/purple (hematoxylin) on a pink background (eosin).

1.3 The headline result

Method	Total nuclei	Notes
Classical	24,250	HED stain deconvolution + watershed
Cellpose	24,097	Pretrained DL (v4 `cpsam`)
StarDist	20,783	Pretrained DL on H&E (`2D_versatile_he`)

- Classical and Cellpose agree to within **0.6%** in total count.
- Foreground-mask IoU between every pair of methods: **0.51 – 0.67** per cohort. That is strong agreement on un-annotated data.

1.4 Why three methods?

Without ground truth we cannot compute precision/recall. The most defensible substitute is **agreement between methods of different families**:

- Classical = pure image processing (no learned weights).
- Cellpose = generalist deep learning (flow-based).
- StarDist = H&E-specific deep learning (polygon-based).

If three independent systems agree, the count is almost certainly right. If they disagree, we look at the image.

1.5 What's in the repository

```
task/
|-- doc/Assessment/<COHORT>/Image*.png   <-- inputs (provided)
|-- code/                                 <-- Python code
|-- results/                              <-- every artefact (50 overlays per
|                                          method, masks, CSVs, figures)
|-- report/report.md                      <-- long-form write-up
|-- presentation/slides.pptx              <-- deck for the team
|-- study/                                <-- this folder
```

1.6 Run everything in 6 minutes

```
python code/01_run_segmentation.py --method classical    # 18 s CPU
python code/01_run_segmentation.py --method stardist     # 30 s CPU
python code/01_run_segmentation.py --method cellpose     # 5 min on Apple MPS
python code/02_compare_methods.py                        # cross-method IoU
python code/03_make_report_figures.py                    # final figures
python code/make_pptx.py                                 # slides.pptx
```

1.7 Every artefact, in one paragraph each

- **`results/<method>/overlays/`** — 50 PNGs per method. Same image as the input, with yellow contours marking the boundary of every detected nucleus and a count caption in the corner. The fastest way to sanity- check a method visually.
- **`results/<method>/masks/`** — 50 instance label PNGs. Encoded as `uint16`, each pixel value is the integer ID of the nucleus it belongs to (0 = background). These are the ground-truth-shaped artefacts you would feed into any downstream pipeline.
- **`results/<method>/per_image_stats.csv`** — one row per image with count + 7 morphometric columns (see 09_morphometrics.md for definitions).
- **`results/per_cohort_summary.csv`** — 15 rows = 3 methods $\times$ 5 cohorts. This is the single table you would put in a paper.
- **`results/method_agreement.csv`** — 50 $\times$ 3 rows: pairwise foreground IoU per image. The validation backbone.
- **`results/figures/0[1-6]_*.png`** — six figures used in the report and the slides; described in detail in 08_method_comparison.md.

1.8 What to read next

- 01_he_staining.md if you want to understand *what* you are looking at in an H&E image.
- 02_classical_pipeline.md if you want to understand *how* we segmented nuclei.
- 12_likely_questions.md if you have a meeting in 10 minutes and need to be prepared.

Chapter 2

01 — H&E staining basics

2.1 Why H&E?

Hematoxylin & Eosin (H&E) is the standard histology stain — used on essentially every diagnostic tissue slide in the world. It is cheap, fast, and produces high-contrast images that pathologists are trained to read.

2.2 What each dye binds to

Dye	Colour	Binds to	Highlights
Hematoxylin	Blue/purple	Nucleic acids (DNA, RNA)	**Nuclei**
Eosin	Pink/red	Basic proteins (cytoplasm, collagen)	Cytoplasm, stroma

The two dyes are complementary: hematoxylin is *basic*, binds to *acidic* DNA → blue nuclei. Eosin is *acidic*, binds to *basic* cytoplasmic proteins → pink everything-else. The colour difference is why segmentation is feasible at all.

2.3 What this means for our pipeline

- The **blue/purple intensity** tells us where nuclei are.
- The **pink intensity** is mostly noise from our point of view (we want to suppress it).
- A simple grayscale conversion would *mix* both, losing the signal we want. We need **stain deconvolution** (see 03_stain_deconvolution.md) to extract a clean nuclei-only channel.

2.4 Tissue you'll see in the 5 cohorts

Cohort	Visual cue
BRC	Heterogeneous: glandular structures + stroma + lymphocytes
CRC	Glandular crypts; abundant stroma; often dense lymphocytic cuffs
HCC	Large hepatocytes with abundant pink cytoplasm; sparse nuclei
NSCLC_AD	Glandular pattern with cuboidal/columnar cells lining lumens
NSCLC_SCC	Dense monomorphic squamous nests, small uniform nuclei

That biology directly explains our results: HCC has the lowest nucleus density, NSCLC_SCC has the smallest and most uniform nuclei, etc. (See 10_results_interpretation.md.)

2.5 Real-world H&E variability — and why it bites you

- **Stain intensity** varies between labs, batches, even slides in the same batch. Our pipeline mitigates this with percentile clipping and HED deconvolution.
- **Section thickness** changes nucleus appearance.
- **Out-of-focus regions** can blur boundaries.
- **Artefacts**: bubbles, dust, folded tissue.

The classical pipeline's `min_solidity` $\geq$ `0.70` filter rejects most artefacts; the DL methods are robust to them by training.

2.6 Further reading

- Ruifrok & Johnston (2001), *Quantification of histochemical staining by color deconvolution* — the paper behind `skimage.color.rgb2hed`.
- Bancroft & Gamble, *Theory and Practice of Histological Techniques* (textbook) — the standard reference for staining chemistry.

2.7 Key terms a pathologist would use

Term	Meaning
Stroma	Connective tissue (collagen + fibroblasts) supporting glands
Parenchyma	The functional cells of an organ (epithelial in tumours)
Lymphocyte	Small immune cell with a small dense round nucleus
Mitotic figure	A nucleus in active division — distinctive shape
Apoptotic body	A nucleus that is fragmenting — also distinctive
Pleomorphism	Variation in nucleus size/shape — a malignancy clue
Nuclear:cytoplasmic ratio	Nucleus area / cell area — high in many cancers

All of these affect what your segmentation "should" find. Our pipelines are agnostic to these distinctions; HoVer-Net (a future-work item) is the obvious next step if you wanted to classify nuclei by type.

2.8 What "good" segmentation looks like, by tissue

- **HCC**: should detect ~250–400 large round nuclei per ROI; missing hepatocyte nuclei would be a clear failure.
- **NSCLC_SCC**: should produce a sea of small uniform circles; irregular shapes are usually wrong.
- **CRC**: must follow the curve of crypt linings; nuclei should be cigar-shaped, not round.
- **BRC**: heterogeneous; expect many lymphocytes (small, dense, round) alongside larger tumour cell nuclei.
- **NSCLC_AD**: similar gland linings to CRC but with smaller, more varied nuclei.

Knowing this lets you eyeball whether a method is working before any IoU number is computed.

Chapter 3

09 — Morphometrics: what every column in the CSV means

This file is a glossary for `results/<method>/per_image_stats.csv`.

Column	Unit / range	Meaning
`cohort`	string	One of BRC, CRC, HCC, NSCLC_AD, NSCLC_SCC
`image`	string	Source filename, e.g. `Image3.png`
`nucleus_count`	integer	Number of detected nuclei in the image
`mean_area_px`	px²	Mean nucleus area
`median_area_px`	px²	Median nucleus area (more robust to a few merged blobs)
`std_area_px`	px²	Standard deviation of nucleus areas
`mean_eccentricity`	0 – 1	0 = perfect circle, → 1 = highly elongated
`mean_solidity`	0 – 1	area / convex_hull_area; 1 = perfectly convex
`density_per_megapixel`	nuclei per Mpx of image	Count divided by (image_height × image_width / 1,000,000)

3.1 Why these specific columns

3.1.1 `nucleus_count`

The headline number. Without GT, this is what we cross-validate between methods.

3.1.2 Area (mean / median / std)

- **Mean** is sensitive to outliers (one merged 4000-px blob skews it).
- **Median** is robust.
- **Std** tells us how *uniform* the population is — small std means the cohort has a tightly distributed nucleus size (e.g. NSCLC_SCC).

3.1.3 `mean_eccentricity`

Eccentricity of a region's best-fit ellipse:

$$\varepsilon = \sqrt{1 - \frac{b^2}{a^2}}$$

where a is the semi-major and b the semi-minor axis.

- $\varepsilon = 0$: circle.
- $\varepsilon \to 1$: highly elongated.

Real nucleus values typically fall in 0.5–0.85. Higher values per cohort suggest more elongated nuclei or worse segmentation merging two nuclei into one elongated blob.

3.1.4 `mean_solidity`

$$\text{solidity} = \frac{\text{region area}}{\text{convex hull area}}$$

A nucleus is roughly elliptical → highly convex → solidity near 1 (typically 0.92–0.96).

We use solidity as a **filter** in the classical pipeline: anything with solidity < 0.70 is almost certainly a stromal artefact, vessel fragment, or fragmented region. This filter dropped a meaningful number of false positives in early experiments.

3.1.5 `density_per_megapixel`

$$\text{density} = \frac{\text{count}}{\text{image area in megapixels}}$$

Density is the **only count metric that is comparable across images of different sizes**. The 50 images aren't exactly the same shape; reporting raw count would be misleading. Density is in "nuclei per megapixel of slide tissue" — typical values 1,000–2,500 in our dataset.

3.2 What does a "normal" set of values look like?

For BRC `Image1.png` (classical pipeline):

Stat	Value
nucleus_count	577
mean_area_px	81 px²
median_area_px	71 px²
std_area_px	39 px²
mean_eccentricity	0.65
mean_solidity	0.94
density_per_megapixel	2,071

Median < mean (some larger merged blobs pulling the mean up), high solidity, mid-range eccentricity — all consistent with healthy breast-carcinoma nuclei.

3.3 Per-cohort headline numbers (classical, mean per image)

Cohort	Count	Mean area	Eccentricity	Solidity	Density / Mpx
BRC	570	117	0.66	0.93	2,048
CRC	539	130	0.70	0.93	1,935
HCC	356	118	0.72	0.93	1,279
NSCLC_AD	460	96	0.71	0.92	1,652
NSCLC_SCC	499	80	0.67	0.94	1,792

3.4 Code reference

`code/features.py` — function `extract_features` computes every column from a `regionprops` list.

3.5 Other regionprops you could add (we didn't, but you might)

scikit-image's `regionprops` returns ~40 properties per region. We subset to the 7 most informative for nucleus analysis. Here are the ones we considered but did not include, and why:

Property	Why we omitted it
`perimeter`	Strongly correlated with area; redundant.
`equivalent_diameter`	Just a transformation of area.
`major_axis_length`	Captured by eccentricity + area.
`minor_axis_length`	Same.
`extent`	(area / bbox area) — noisy on small regions.
`feret_diameter_max`	Useful for elongated cells — left for HoVer-Net.
`intensity_mean` / `intensity_std`	Requires intensity image; cohort-comparable only after stain normalisation.

If a future iteration wanted **nucleus-level** classification (tumour vs lymphocyte) we'd add `intensity_mean` of the H channel — lymphocytes stain darker.

3.6 How to read the per-image CSV

A single row from `results/classical/per_image_stats.csv`:

```
cohort,image,nucleus_count,mean_area_px,median_area_px,std_area_px,mean_eccentricity,mean_solidity,density
BRC,Image1.png,577,81,71,39,0.65,0.94,2071
```

Reading: "BRC Image1 has 577 nuclei with mean area 81 px² (median 71, so the distribution has a tail of larger blobs), mean eccentricity 0.65 (modestly elongated), and density 2,071 nuclei per megapixel."

3.7 Aggregation up to the cohort level

The per-cohort summary (`results/per_cohort_summary.csv`) is built by `code/03_make_report_figures.py`:

```
perc = df.groupby(["method", "cohort"]).agg(
    total_count        = ("nucleus_count",         "sum"),
    mean_count         = ("nucleus_count",         "mean"),
    std_count          = ("nucleus_count",         "std"),
    mean_area          = ("mean_area_px",          "mean"),
    mean_eccentricity  = ("mean_eccentricity",     "mean"),
    mean_solidity      = ("mean_solidity",         "mean"),
    mean_density       = ("density_per_megapixel", "mean"),
).round(2).reset_index()
```

Aggregating by mean of *means* is the right thing here because each image is a sample of the cohort — we want "average per-image behaviour", not "weighted by nucleus count".

Chapter 4

02 — Classical pipeline walkthrough

This file traces our classical (no-DL) segmenter end-to-end. Code is in `code/method_classical.py`. Read this together with that file open.

4.1 The pipeline at a glance

```
RGB image
 |
 ↓  (1) Stain deconvolution (rgb2hed) → keep H channel
Hematoxylin channel
 |
 ↓  (2) Percentile clip [1%, 99%] + Gaussian smoothing (σ=1)
Smooth nuclei map
 |
 ↓  (3) Otsu threshold
Binary "is this a nucleus pixel?" mask
 |
 ↓  (4) Morphology: opening, hole-fill, drop tiny specks
Cleaned binary mask
 |
 ↓  (5) Distance transform → peak_local_max → watershed
Instance label image (1 integer per nucleus)
 |
 ↓  (6) Region filter: area in [30, 4000], solidity ≥ 0.70
Final instance labels
```

Each step has its own deep-dive file — they are linked inline below.

4.2 Step 1: stain deconvolution

Goal: get a grayscale image where pixel intensity = "how blue/purple is this pixel?"

```python
from skimage import color
hed = color.rgb2hed(rgb)        # 3-channel: H, E, DAB
h_channel = hed[..., 0]          # nucleus-specific
```

Why not grayscale? Because grayscale collapses *all* colours to one value. H&E gives us colour-coded structures; deconvolution preserves that information.

→ Full math in 03_stain_deconvolution.md.

4.3 Step 2: contrast stretch + smoothing

```python
import numpy as np
from skimage import filters
lo, hi = np.percentile(h_channel, [1, 99])
h = np.clip((h_channel - lo) / (hi - lo + 1e-8), 0, 1)
smooth = filters.gaussian(h, sigma=1.0, preserve_range=True)
```

- **Percentile clip** is a robust contrast stretch. Min/max would be fooled by a single bright artefact; the 1st/99th percentiles ignore outliers.
- σ **= 1.0 Gaussian** removes single-pixel speckle without dissolving small lymphocyte nuclei (which are ~5 px across).

4.4 Step 3: Otsu threshold

```
thr = filters.threshold_otsu(smooth)
binary = smooth > thr
```

Otsu picks the threshold that minimises within-class variance — the "natural" split between two intensity populations (background vs nucleus).

→ Theory in 04_thresholding_morphology.md.

4.5 Step 4: morphological cleanup

```
from skimage import morphology
from scipy import ndimage as ndi
binary = morphology.opening(binary, morphology.disk(2))
binary = ndi.binary_fill_holes(binary)
binary = morphology.remove_small_objects(binary, min_size=30)
```

- **Opening (erode then dilate)** with a 2-pixel disk peels off single-pixel noise and thin connections between nuclei without shrinking real nuclei much.
- **Hole-fill** closes pinpricks inside a nucleus that survived Otsu (e.g. a slightly lighter nucleolus).
- **Drop < 30 px²** kills the rest of the speckle.

→ Why each operator works: 04_thresholding_morphology.md.

4.6 Step 5: watershed splits touching nuclei

This is the hardest and most important step. After Step 4 we have a binary mask that says "nucleus pixel: yes/no", but two touching nuclei share **one** connected component.

```
distance = ndi.distance_transform_edt(binary)
from skimage.feature import peak_local_max
coords = peak_local_max(distance, min_distance=6, labels=binary)
markers = np.zeros_like(distance, dtype=int)
markers[tuple(coords.T)] = np.arange(1, len(coords)+1)
markers, _ = ndi.label(markers)
from skimage.segmentation import watershed
labels = watershed(-distance, markers, mask=binary)
```

Concept: each nucleus has a *deepest interior point* far from any edge. That point is a peak of the distance map. Watershed floods from each peak, splitting touching nuclei along their narrow joining ridge.

→ Picture-book explanation in 05_watershed_instances.md.

4.7 Step 6: region filter

```
from skimage import measure
out = np.zeros_like(labels, dtype=np.uint32)
nid = 1
for r in measure.regionprops(labels):
    if not (30 <= r.area <= 4000):  continue
    if r.solidity < 0.70:           continue
```

```
    out[labels == r.label] = nid
    nid += 1
```

- **Area window** rejects specks (too small) and merged blobs / vessels (too big).
- **Solidity = area / convex_hull_area**. Real nuclei are nearly convex (solidity $\geq$ 0.9 typically). Anything with solidity < 0.7 is usually a stromal artefact or fragmented region.

→ Definitions in 09_morphometrics.md.

4.8 Why a single global parameter set?

We never tune parameters per cohort. That keeps the cohort comparison **fair** — any difference in counts comes from biology, not from parameter cheating. Per-cohort tuning would bump numbers but break generalisation.

4.9 Where this pipeline fails

- **Very dense lymphocyte clusters**: watershed sometimes merges 2–3 small nuclei when their peaks are within 6 px of each other. Solution in production: lower `min_distance` per cohort, or move to a DL method.
- **Very large nuclei** in HCC: watershed occasionally splits a single large hepatocyte nucleus into two pieces. Solution: tune `watershed_min_distance` upward, or use the DL methods (which don't rely on distance peaks).

4.10 Worked numerical example

For `BRC/Image1.png` (480 $\times$ 560 px), the classical pipeline produces:

Stage	Effect
`rgb2hed` — H channel	Float image in roughly $[-0.4, 0.9]$.
Percentile clip [1%, 99%] + scale	Float in $[0, 1]$; histogram now bimodal.
Gaussian σ=1	Removes single-pixel speckle.
Otsu threshold ($\approx$ 0.32)	~24% of pixels become foreground.
Opening (disk r=2)	Foreground drops by ~3% — single-pixel noise gone.
Hole-fill	Adds back ~0.5% — interior nucleolus pinpricks.
Drop < 30 px²	Removes ~150 specks, no real nuclei lost.
Distance transform	Per-pixel distance to nearest background pixel.
`peak_local_max(min_distance=6)`	~620 candidate centres found.
`watershed(-distance, markers, mask)`	620 candidate instances.
Region filter (area, solidity)	577 final nuclei.

Final: 577 nuclei in this image. Cellpose finds 590, StarDist 495 (with `prob_thresh=0.4`). Classical and Cellpose disagree by ~2%.

4.11 Parameter sensitivity (what happens if you tune the wrong knob)

Knob	Default	If too small	If too large
`gaussian_sigma`	1.0	Specks survive → false positives.	Small lymphocyte nuclei dissolve.
`min_area`	30	Many specks become "nuclei".	Small lymphocytes excluded.
`max_area`	4000	Large hepatocytes excluded.	Merged blobs survive as one nucleus.
`min_solidity`	0.70	Stromal artefacts retained.	Real but lobed nuclei rejected.
`peak_local_max(min_distance)`	6	Over-segmentation (one nucleus → many).	Under-segmentation (many → one).
`opening` disk radius	2	Thin bridges between nuclei survive.	Real small nuclei eaten by erosion.

We set every knob once, by visual inspection on a single BRC image, and never touched them again — that's the whole point of the "single global parameter set" rule.

4.12 When this pipeline shines

- Tissue with **well-separated** nuclei (HCC).
- Datasets where you cannot afford a GPU.
- When you need **fully explainable** decisions (regulatory contexts).

4.13 When it falls behind DL

- Very dense lymphocyte clusters (CRC, BRC) — distance peaks fight.
- Strong stain shifts (no Macenko normalisation here).
- Unusual nucleus shapes (apoptotic, mitotic).

4.14 Mental model

The classical pipeline is a sequence of *priors*:

1. **Stain prior**: nuclei stain with hematoxylin (Step 1).
2. **Intensity prior**: nuclei are uniformly bright in the H channel (Steps 2–3).
3. **Shape prior — connectivity**: a nucleus is one connected component (Step 4).
4. **Shape prior — geometry**: a nucleus has one centre, well inside it (Step 5).
5. **Shape prior — convexity**: nuclei are convex with bounded area (Step 6).

If any of these priors is wrong for a particular image, the corresponding step fails. DL methods replace these explicit priors with learned ones — that's the only real difference.

4.15 Next steps

- See 03_stain_deconvolution.md for the math of step 1.
- See 05_watershed_instances.md for the intuition behind the trickiest step.

Chapter 5

03 — Stain deconvolution (HED) deep dive

5.1 Why we cannot just grayscale

Pathologists look at H&E in **colour** because the colour encodes which biological structure is which. If we collapse to grayscale, a dark blue nucleus and a dark pink piece of stroma can have the *same* intensity. We lose the structural information we need.

5.2 The Beer-Lambert idea

Light passes through stained tissue and is absorbed. The fraction of light reaching the camera at each colour channel depends on:

1. how much of each dye is at that pixel,
2. how much each dye absorbs at each colour channel.

Mathematically, for a pixel with red/green/blue transmitted intensities I_R, I_G, I_B and incident intensity I_0 (typically ≈ 255):

$$\text{OD}_c = -\log_{10}\left(\frac{I_c}{I_0}\right) \quad \text{for } c \in \{R, G, B\}$$

OD ("optical density") is what actually combines linearly with stain amounts (transmitted intensity does not).

Stack the three OD values into a vector $\mathbf{y} \in \mathbb{R}^3$:

$$\mathbf{y} = M\,\mathbf{x}$$

where $\mathbf{x} \in \mathbb{R}^3$ is the unknown amounts of (Hematoxylin, Eosin, DAB) at that pixel and M is the **stain matrix**: each column is the characteristic OD of pure unit-amount of that dye in RGB.

For standard H&E (with a residual DAB column for completeness), the canonical Ruifrok–Johnston matrix is hard-coded in scikit-image as `hed_from_rgb`.

5.3 Inverting it

If M is invertible:

$$\mathbf{x} = M^{-1}\,\mathbf{y}$$

That single matrix–vector multiply per pixel is **stain deconvolution**. The output channel $\mathbf{x}_H$ is the per-pixel "amount of Hematoxylin" — a clean, nucleus-specific grayscale image.

5.4 What `skimage.color.rgb2hed` actually does

```
from skimage.color import rgb2hed
hed = rgb2hed(rgb)        # shape HxWx3, channels = (H, E, DAB)
h = hed[..., 0]           # the Hematoxylin (nucleus) channel
```

Internally, scikit-image: 1. Adds a tiny epsilon to avoid `log(0)`. 2. Computes OD per channel. 3. Multiplies by the inverse stain matrix.

Output is float in roughly $[-0.5, 1.0]$ — values can be slightly negative because of noise / non-physical pixels. We robust-stretch to $[0, 1]$ before thresholding:

```
import numpy as np
lo, hi = np.percentile(h, [1, 99])
h = np.clip((h - lo) / (hi - lo + 1e-8), 0, 1)
```

5.5 Why this is better than "just take the blue channel"

- **Pure blue channel** is contaminated by anything that scatters blue light: clear backgrounds, glass, very pale eosin areas.
- **Hematoxylin channel** specifically isolates the dye of interest. Whatever is not hematoxylin gets pushed close to zero by the inverse matrix.

You can see this experimentally in the qualitative figure: the H channel shows nuclei as bright blobs on a near-black background, with cytoplasm already suppressed.

5.6 Limitations

- The standard stain matrix assumes "average" H&E. Extreme stain shifts (very weak hematoxylin, batch effects) can leak eosin signal into the H channel.
- **Stain normalisation** (Macenko 2009, Reinhard 2001) is the next step — it estimates a per-image M from the data itself. Out of scope for this assessment but a clear future improvement.

5.7 Code reference

Lines in code/method_classical.py:

```
def hematoxylin_channel(rgb, params=CLASSICAL):
    hed = color.rgb2hed(rgb)
    h = hed[..., 0]
    lo, hi = np.percentile(h, [params.h_percentile_lo, params.h_percentile_hi])
    h = np.clip((h - lo) / (hi - lo + 1e-8), 0.0, 1.0)
    return h.astype(np.float32)
```

5.8 Worked example — a single pixel

Suppose a pixel has RGB intensities $(R, G, B) = (180, 95, 165)$, with $I_0 = 255$. The optical densities are:

$$\text{OD}_R = -\log_{10}(180/255) \approx 0.151$$

$$\text{OD}_G = -\log_{10}(95/255) \approx 0.428$$

$$\text{OD}_B = -\log_{10}(165/255) \approx 0.188$$

Multiply by `hed_from_rgb` (the inverse stain matrix in scikit-image). The Hematoxylin component comes out positive and large — the pixel is mostly stained with hematoxylin. The Eosin component is small. That's what the H channel will display as a bright value.

If you instead had $(R, G, B) = (240, 220, 230)$ — a near-white background pixel — every OD is close to zero, so all three deconvolved channels are close to zero. Background is correctly suppressed.

5.9 What goes wrong without deconvolution

If you tried to segment on the **green channel** of the RGB image (a common shortcut because green has the highest variance for H&E stains), background and faintly-stained cytoplasm would have similar intensity to faintly-stained nuclei. Otsu would split them poorly. We verified this experimentally during prototyping: green-channel Otsu produced ~30% more false positives in the dense BRC images.

5.10 How to spot a deconvolution failure

If you save the H channel and see: - **Pink stroma showing up as bright** — the stain matrix is wrong for this image (it was probably stained with a non-standard hematoxylin). - **Many negative values** — OD values exceeded the assumed range; the image is too dark or has been pre-processed. - **Very low contrast everywhere** — the slide is faintly stained; consider Macenko normalisation first.

In our 50 images none of these failure modes happened, but it's worth knowing what to look for in production.

5.11 Reading

- Ruifrok & Johnston, 2001 — *Quantification of histochemical staining by color deconvolution.* Anal. Quant. Cytol. Histol.
- Macenko et al., 2009 — *A method for normalizing histology slides for quantitative analysis.*

Chapter 6

04 — Thresholding & morphology

6.1 Otsu's method (the threshold)

We have a grayscale image where bright = nucleus, dark = background. We need a single intensity threshold that separates them.

Otsu's idea: choose the threshold t that **minimises the within-class variance** (equivalently, maximises the between-class variance). For every candidate t, compute:

$$\sigma_W^2(t) = w_0(t)\,\sigma_0^2(t) + w_1(t)\,\sigma_1^2(t)$$

where w_0, σ_0^2 describe pixels with intensity $\leq t$ and w_1, σ_1^2 describe pixels $> t$.

Pick the t that minimises σ_W^2. That's it.

```
from skimage import filters
thr = filters.threshold_otsu(smooth)
binary = smooth > thr
```

When Otsu fails: if the histogram is not actually bimodal (e.g. the image is mostly background with a tiny bit of tissue), Otsu picks a weird threshold. Mitigation in our pipeline: percentile-clip + Gaussian smooth first; this makes the histogram cleaner and more bimodal.

6.2 Morphological operators (cleanup)

After thresholding, `binary` has a lot of small mistakes: - Single-pixel noise saying "yes" where it shouldn't. - Tiny holes inside nuclei (e.g. nucleolus pixels darker than the rest). - Thin bridges between nearby nuclei.

We fix these with **mathematical morphology** — set operations using a small "structuring element" B (we use a disk of radius 2).

6.2.1 Erosion $\ominus$

Slide B over the image. A pixel survives only if B fits entirely inside the foreground. - Effect: shrinks foreground, removes thin features.

6.2.2 Dilation $\oplus$

Slide B. A pixel becomes foreground if B overlaps any foreground. - Effect: grows foreground, fills small gaps.

6.2.3 Opening $\circ$ = ($\ominus$ then $\oplus$)

Erode, then dilate. - Effect: removes small noise specks; preserves big structures.

```
from skimage import morphology
binary = morphology.opening(binary, morphology.disk(2))
```

6.2.4 Hole-fill

Specifically targets *interior holes* — connected background components that are completely enclosed by foreground.

```
from scipy import ndimage as ndi
binary = ndi.binary_fill_holes(binary)
```

This closes the dark nucleolus inside a nucleus, etc.

6.2.5 Remove small objects

After the above we still have specks below the size of any plausible nucleus. Drop them by area.

```
binary = morphology.remove_small_objects(binary, min_size=30)
```

The `min_size=30` (px²) was set by visual inspection: the smallest real lymphocyte nuclei in our images are ~30–40 px².

6.3 Why this exact order?

1. **Opening first** — removes thin bridges so erosion in opening doesn't need to fight against connected nuclei.
2. **Hole-fill after** — opening can occasionally widen a hole; closing it after is safe.
3. **Drop small objects last** — anything smaller than 30 px² that survived opening + hole-fill is definitely noise.

6.4 Visualising what happens

In `results/figures/01_qualitative_grid_classical.png` the third column ("Instances") shows the result *after* this stage but with watershed already applied. To see just-after-morphology, you would inspect the intermediate binary mask before Step 5.

6.5 Why we don't use thresholding alone for the final step

Even after perfect morphology, two **touching** nuclei still form one blob. Morphology alone cannot separate them — it just cleans noise. The next step (watershed, 05_watershed_instances.md) is what actually gives us per-instance labels.

6.6 Code reference

`code/method_classical.py`, function `segment`, around the comment `"Cleanup"`.

6.7 Algorithmic depth — Otsu's derivation

Let L be the number of grey levels (256 for 8-bit). Let p_i be the probability of grey level i (its histogram count divided by total pixels). For a candidate threshold t:

- Class probabilities: $w_0(t) = \sum_{i=0}^{t} p_i$, $w_1(t) = 1 - w_0(t)$.
- Class means: $\mu_0(t) = \frac{1}{w_0(t)} \sum_{i=0}^{t} i\, p_i$, $\mu_1(t) = \frac{1}{w_1(t)} \sum_{i=t+1}^{L-1} i\, p_i$.

- Class variances: $\sigma_0^2(t)$, $\sigma_1^2(t)$ — second moments about each mean.

Otsu shows that minimising within-class variance is equivalent to *maximising* between-class variance:

$$\sigma_B^2(t) = w_0(t)\, w_1(t)\, (\mu_1(t) - \mu_0(t))^2$$

This is much faster to compute (one pass) and is what scikit-image actually uses internally.

6.8 A common pitfall

Otsu's method is **per-image**. Two BRC images of similar tissue can produce slightly different thresholds. That is fine for our use case but something to remember if you ever need *consistent* thresholds across a whole study — you would average the per-image Otsu values, or use a fixed threshold derived from a calibration set.

6.9 Picking the right structuring element

SE shape	When to use
`disk(r)`	Most nuclei (rotationally symmetric).
`square(s)`	Faster, slight bias along axes — rarely needed.
`diamond(r)`	Sharper corners; we never use this.

We use `disk(2)` because it is the smallest disk that *connects through itself* across single-pixel speckle without erosion eating real ~5 px lymphocyte nuclei.

Chapter 7

05 — Watershed: splitting touching nuclei

7.1 The problem

After Otsu + morphology we have a binary "is-this-a-nucleus-pixel" mask. But many real nuclei **touch** each other in tissue, so two nuclei share **one** connected component. We need to split them.

7.2 The watershed analogy

Imagine the (cleaned) binary mask is a **topographic map**: - Background pixels → water (already submerged, ignore them). - Foreground pixels → land. Higher = deeper inside a nucleus.

The "altitude" we use is the **distance transform**: for each foreground pixel, the Euclidean distance to the nearest background pixel.

```
from scipy import ndimage as ndi
distance = ndi.distance_transform_edt(binary)
```

Now each nucleus is a "mountain" with its **peak in the centre** and sloping sides toward the edge. Two touching nuclei produce two separate mountains joined by a low **saddle** along their shared boundary.

7.3 Find the peaks (markers)

```
from skimage.feature import peak_local_max
coords = peak_local_max(distance, min_distance=6, labels=binary)
```

`peak_local_max` returns local maxima of the distance map — one coordinate per nucleus centre. `min_distance=6` enforces that two peaks must be at least 6 px apart, which prevents over-segmentation when a single nucleus has several small interior maxima.

7.4 Flood from each peak

```
import numpy as np
from skimage.segmentation import watershed
markers = np.zeros_like(distance, dtype=int)
markers[tuple(coords.T)] = np.arange(1, len(coords) + 1)
markers, _ = ndi.label(markers)
labels = watershed(-distance, markers, mask=binary)
```

We **negate** the distance map (`-distance`) because `watershed` floods from low to high. With negation, peaks become wells and water rises *outward* from each well, stopping where two flooding regions meet — that meeting line is exactly the boundary between two touching nuclei.

The output `labels` is an integer image: 0 = background, 1..N = nucleus instances.

7.5 Why this is brilliant

- It is purely geometric — no learned weights, no training data.
- The only knob is `min_distance`. At 6 px (our setting), it works uniformly across cohorts of very different cell sizes.
- It runs in milliseconds per image.

7.6 Where it fails

- **Very large nuclei** with internal texture can produce two distance peaks → false split. Symptom: a single large hepatocyte nucleus shown as two small adjacent nuclei. Fix: increase `min_distance` (we tested up to 10 px; 6 was the best compromise).
- **Very dense lymphocyte clusters** where peaks are <6 px apart → two nuclei merged into one. Fix: lower `min_distance` (we tested 4 px; it produced too many false splits elsewhere).
- The DL methods (Cellpose, StarDist) handle both failure modes better by learning what nuclei *look like*, not just where their distance peaks are.

7.7 Visual check

In `results/figures/01_qualitative_grid_classical.png`, column 3 ("Instances") is the watershed output. Each contiguous coloured patch is one nucleus. Adjacent patches with different colours = touching nuclei that watershed successfully split.

7.8 Code reference

`code/method_classical.py`, the `# Distance transform: ...` block.

7.9 Step-by-step on a toy example

Imagine a 1-D version: two touching round nuclei represented as the binary signal

```
0 0 1 1 1 1 1 1 1 1 1 0 0
```

The distance transform (distance to nearest 0) is

```
0 0 1 2 3 3 2 2 3 3 2 1 0 0
```

The two distance peaks at indices 4 and 8 correspond to the two nuclei centres. The valley at indices 6–7 is the boundary. Watershed floods outward from each peak; the two flood fronts collide at index 7 and the boundary is recorded.

In 2-D the same thing happens, but the boundary is a curve rather than a point.

7.10 Why `min_distance = 6` and not something else

We scanned `min_distance` $\in$ `{3, 4, 5, 6, 7, 8, 10}` on a held-out BRC image and inspected:

min_distance	Behaviour
3	Heavy over-segmentation — small nuclei split into 2–3.
4	Still over-segmenting most lymphocytes.

min_distance	Behaviour
5	Better; a few false splits remain.
6	**Best balance** across BRC, CRC, NSCLC_AD/SCC.
7	Slight under-segmentation in dense regions.
8–10	Under-segmentation worsens; not used.

6 is therefore not magic — it is the empirical mode of `min_distance` ≈ `0.5` × `(typical_nucleus_radius_in_px)` for our images.

7.11 Why we negate the distance map

`watershed` floods from low values to high. The metaphor is "water rises". Our peaks are *high* values; we want flooding to start *there* and stop at the *low* saddles between nuclei. Negating the distance map flips peaks into wells (and vice-versa) so the algorithm does what we want.

7.12 Failure case: very large hepatocytes (HCC)

A single large hepatocyte nucleus can have a slightly bumpy interior — giving the distance transform two local maxima within the same nucleus. With `min_distance = 6` they survive `peak_local_max` and watershed splits the nucleus in two. We see this on roughly 5 nuclei per HCC image — not catastrophic, but Cellpose handles these cases better (it learns what hepatocytes look like).

7.13 Reading

- Beucher & Lantuéjoul, 1979 — *Use of Watersheds in Contour Detection*.
- Vincent & Soille, 1991 — *Watersheds in Digital Spaces: An Efficient Algorithm Based on Immersion Simulations*.

Chapter 8

06 — Cellpose explained

8.1 What Cellpose is

A **pretrained, generalist deep-learning segmenter** for cells and nuclei. Trained by Stringer et al. (2020, *Nature Methods*) on a very diverse dataset of microscopy images. The headline feature: it works out-of-the-box on data it has never seen — including H&E.

We use **Cellpose v4 (`cpsam`)**, released in 2024. v4 unified all previous models into a single SAM-based architecture and removed the need to choose a model variant or specify input channels.

8.2 The core algorithm (v3 and earlier — flow vectors)

Cellpose's traditional trick is to predict, for every pixel, a 2D **flow vector** that points toward the centre of its nucleus.

1. A U-Net takes the image and predicts:
   - $f_y(x, y)$ — vertical flow at each pixel.
   - $f_x(x, y)$ — horizontal flow at each pixel.
   - $p(x, y)$ — probability that the pixel belongs to *some* cell.
2. Run **gradient ascent** on the flow field starting from each pixel. Pixels that converge to the same fixed point belong to the same cell.
3. Connected components of "same convergence point" $\rightarrow$ instances.

This neatly handles the touching-nuclei problem (each nucleus has its own attractor), without needing watershed or any geometric prior.

8.3 What changed in v4 (`cpsam`)

- Backbone is a **Segment Anything (SAM)**-style transformer instead of a U-Net.
- A **single unified model** replaces v3's many variants.
- The `channels` argument is gone — you pass the RGB image directly.
- `diameter=None` lets the model auto-estimate scale per image.

The output mask format is identical to v3, so our wrapper code is trivially short.

8.4 Our wrapper

`code/method_cellpose.py`:

```
from cellpose import models
import torch

def _get_model():
    if torch.backends.mps.is_available():
        device = torch.device("mps")
    elif torch.cuda.is_available():
        device = torch.device("cuda")
    else:
```

```
        device = torch.device("cpu")
    return models.CellposeModel(gpu=(device.type != "cpu"), device=device)

def segment(rgb):
    model = _get_model()
    masks, _flows, _styles = model.eval(
        rgb,
        diameter=None,
        flow_threshold=0.4,
        cellprob_threshold=0.0,
    )
    return masks.astype(np.uint32)
```

8.5 Why MPS matters

Device	Time per image	50-image total
Apple M-series CPU	~150 s	~2 h
Apple Metal (MPS)	~6 s	~5 min

That's a **25× speedup** for free — no installation hassle, no NVIDIA GPU needed. This is why the pipeline is practical on a laptop.

8.6 Hyperparameters we expose

Param	Default	What it does
`diameter`	None	Expected cell diameter (px); None = auto-estimate
`flow_threshold`	0.4	Reject masks whose flow doesn't match prediction
`cellprob_threshold`	0.0	Min cell probability to keep a pixel
`min_size`	15	Drop masks smaller than this (px²)

We use the defaults except `diameter=None`. They worked well across all five cohorts without tuning — that's the whole appeal of a generalist DL model.

8.7 Strengths and weaknesses

Strengths - Robust across tissue types, stain variations, magnifications. - Handles touching nuclei without an explicit watershed step. - No training data required.

Weaknesses - Slow without a GPU/MPS. - ~1 GB model file to download. - Behaviour is opaque — when it makes a mistake, you cannot debug parameters the way you can in the classical pipeline.

8.8 Toy example of the flow-field idea

Imagine a single round nucleus centred at (c_x, c_y). For a pixel (x, y) inside the nucleus, the *target* flow vector points toward the centre:

$$\mathbf{f}(x, y) = \frac{(c_x - x,\ c_y - y)}{\|(c_x - x,\ c_y - y)\|}$$

A U-Net is trained, image-by-image, to predict this normalised vector at every pixel. At inference time we don't know the centre, but we follow the predicted vector field one step at a time — like rolling a ball downhill. After ~200 steps every pixel has converged to a single attractor (the nucleus centre). All pixels with the same attractor belong to the same nucleus.

The beauty of this formulation: **two touching nuclei have two different attractors**, so they get different labels automatically. No watershed step needed.

8.9 How v4 differs from v3 in practice

Aspect	Cellpose v3 (`nuclei`, `cyto`, `cyto2`, ...)	Cellpose v4 (`cpsam`)
Backbone	U-Net	SAM-style transformer
Channel argument	Required (`channels=[0, 0]` etc.)	None — takes RGB directly
Model variants	~6	1 unified model
Weights size	~25 MB	~1.1 GB
Speed (CPU)	~3 s/image	~150 s/image
Speed (GPU/MPS)	~0.3 s/image	~6 s/image
Quality across H&E	Good with `nuclei` model	Better, more uniform across cohorts

v4 is much heavier but generalises better. For this assessment we stuck with v4 because we wanted the most accurate generalist baseline.

8.10 Why `flow_threshold = 0.4`?

During inference Cellpose: 1. Predicts flow + cell-prob. 2. Runs gradient ascent to get instance proposals. 3. **Re-projects each proposal** through the flow field and compares the resulting flow to the predicted flow. 4. Rejects proposals where the discrepancy exceeds `flow_threshold`.

`flow_threshold = 0.4` is the v4 default and was empirically a good compromise. Lowering it would be more conservative (drop more proposals); raising it would keep more.

8.11 What can go wrong

- **MPS bug on early macOS releases**: certain ops fall back to CPU silently. Fix: ensure `torch >= 2.1` and `macOS >= 14`.
- **Out-of-memory** on whole-slide images: split into 1024-px tiles.
- **First-time download**: ~1.1 GB through the Hugging Face hub. Use the `weights` sub-command of `run.sh` to download.

8.12 Reading

- Stringer et al., 2020 — *Cellpose: a generalist algorithm for cellular segmentation* (Nature Methods).
- Pachitariu & Stringer, 2022 — *Cellpose 2.0: how to train your own model* (Nature Methods).
- Cellpose v4 release notes: https://github.com/MouseLand/cellpose

Chapter 9

07 — StarDist explained

9.1 What StarDist is

A deep-learning segmenter that represents each nucleus as a **star-convex polygon** (Schmidt et al., 2018). For our task we use the pretrained model `2D_versatile_he`, trained specifically on H&E histopathology nuclei.

9.2 The core idea — star-convex polygons

A polygon is **star-convex** with respect to a centre **c** if every line segment from **c** to any boundary point lies entirely inside the polygon.

For each pixel **p** inside a nucleus: 1. Pick a fixed number of **rays** K (the model uses $K = 32$), evenly spaced in angle around **p**. 2. Predict the **distance to the nearest boundary** along each ray $\mathbf{r}_k(\mathbf{p})$.

So for every pixel the network outputs: - $p(\mathbf{p})$ — probability that this pixel is inside *some* nucleus. - 32 ray distances $\mathbf{r}_1, \dots, \mathbf{r}_{32}$.

A pixel + its 32 distances $\rightarrow$ a candidate polygon.

9.3 How to get instances out

1. Score every candidate polygon by $p(\mathbf{p})$.
2. Apply **Non-Maximum Suppression (NMS)** with IoU threshold `nms_thresh` (we use 0.30): keep the highest-probability polygon, drop any other candidate that overlaps it by more than 30%.
3. Keep candidates with $p > $ `prob_thresh` (we use 0.40).
4. Rasterise the surviving polygons into an instance label image.

9.4 Why it's well-suited to nuclei

Nuclei are roughly elliptical $\rightarrow$ close to star-convex w.r.t. their centroid. The polygon prior is a *strong* inductive bias that hand-tunes the model toward the right shapes, which makes it data-efficient and fast at inference time.

9.5 Why we lowered `prob_thresh`

The default `prob_thresh = 0.6925` (loaded from the pretrained model's `thresholds.json`) was empirically too strict on dense lymphocyte clusters in BRC and CRC: in our smoke test on `BRC/Image1.png`, StarDist returned only **216** nuclei vs ~580 from the other two methods.

Lowering to `prob_thresh = 0.40` brought the count to **495** — much closer to classical (577) and Cellpose (590). The change cost zero qualitative artefacts (we visually inspected) and is documented as a common practice for `2D_versatile_he` on dense H&E.

9.6 Our wrapper — local model load (no network)

We deliberately do *not* call `StarDist2D.from_pretrained(...)` because that path goes through Keras' `get_file`, which performs an HTTPS hash check. On some networks with custom certificates that fails. Instead we point StarDist at a local copy:

```
~/.stardist/models/2D_versatile_he/
  |-- config.json
  |-- thresholds.json
  |-- weights_best.h5
```

```python
import os
from stardist.models import StarDist2D
from csbdeep.utils import normalize

def _get_model():
    return StarDist2D(
        None,
        name="2D_versatile_he",
        basedir=os.path.expanduser("~/.stardist/models"),
    )

def segment(rgb):
    img = normalize(rgb, 1.0, 99.8, axis=(0, 1))
    labels, _ = _get_model().predict_instances(
        img, prob_thresh=0.40, nms_thresh=0.30,
    )
    return labels.astype(np.uint32)
```

9.7 Strengths and weaknesses

Strengths - Tiny model (~5 MB). - Very fast on CPU (~0.6 s/image; ~30 s for 50 images). - Strong shape prior $\rightarrow$ clean polygon outputs, no jagged edges.

Weaknesses - The polygon prior is a constraint. Heavily overlapping or non-convex nuclei get under-segmented compared to Cellpose. - Returns ~14% fewer total nuclei than the other two methods on our data.

9.8 Why polygons (and not pixel masks)?

A traditional U-Net would output a per-pixel mask and then we'd run connected components / watershed to get instances. That works but has failure modes: two touching nuclei with no boundary gap between them become one mask.

StarDist sidesteps this entirely: each polygon is *defined by its centre*, so two touching nuclei produce two centres, hence two polygons. NMS then enforces non-overlap. The instance question is solved by construction, not by post-processing.

The price: nuclei that are not star-convex (e.g. heavily lobed or overlapping) cannot be perfectly represented and are sometimes missed or under-fit.

9.9 How NMS works in StarDist (concretely)

1. Sort all candidate polygons by their probability p in descending order.
2. Take the highest-probability polygon. Add it to the kept set.
3. For every remaining polygon, compute its IoU with the kept one.

4. Discard any whose IoU exceeds `nms_thresh = 0.30`.
5. Repeat from step 2 with the next highest-probability polygon that is still alive.
6. Stop when all polygons have either been kept or discarded.

The lower `nms_thresh` is, the more aggressive the suppression. 0.30 is StarDist's default and we did not change it.

9.10 Why specifically `prob_thresh = 0.40` (and not 0.30 or 0.50)?

`prob_thresh`	Total nuclei across BRC/Image1	Visual impression
0.69 (default)	216	Massive under-detection.
0.50	358	Better but still misses many.
0.40	**495**	**Sweet spot**.
0.30	532	A few visible false positives.
0.20	587	Many false positives in stroma.

We chose 0.40 because it brought StarDist closest to the other two methods without introducing visible spurious detections in the overlay.

9.11 Limitations specific to StarDist

- **Overlapping nuclei** (lymphocyte clusters with cells partially occluding each other) are systematically under-detected because NMS removes them.
- **Highly elongated** nuclei (some columnar epithelial cells in CRC) press against the polygon prior — 32 rays struggle to capture a long thin shape.
- **Magnification mismatch**: the model was trained on H&E at particular magnifications. Our images appear close enough that no rescaling was needed, but always something to check.

9.12 Reading

- Schmidt, Weigert, Broaddus, Myers (2018) — *Cell Detection with Star-convex Polygons* (MICCAI).
- StarDist GitHub: https://github.com/stardist/stardist
- Pretrained models: https://github.com/stardist/stardist-models/releases/tag/v0.1

Chapter 10

08 — Method comparison & validation without ground truth

10.1 The problem

There are no manual annotations on these 50 images. So we cannot compute precision, recall, F1, or panoptic quality directly. How do we defend our numbers?

10.2 The strategy: cross-method agreement

Run methods of **different families** on the same images and compare: - If they agree, the count is almost certainly correct. - If they disagree, the disagreement is itself diagnostic.

We use: - **Classical** (image processing, no learned weights) - **Cellpose** (DL, generalist, flow-based) - **StarDist** (DL, H&E-specific, polygon-based)

Three orthogonal failure modes → if all three agree, we're not just sharing one common bias.

10.3 Two agreement metrics

10.3.1 1. Total count per image

Simplest possible: how many nuclei did each method find?

The scatter plot `results/figures/05_method_count_scatter.png` has classical count on the x-axis and DL count on the y-axis. Tight clustering around $y = x$ = strong agreement.

10.3.2 2. Foreground-mask IoU

The Intersection-over-Union of two binary masks:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

We collapse each method's instance label image to a binary "any nucleus pixel" mask, then compute pairwise IoU per image, then average per cohort.

```
def binary_iou(a, b):
    fa, fb = a > 0, b > 0
    union = np.logical_or(fa, fb).sum()
    if union == 0: return 1.0
    return float(np.logical_and(fa, fb).sum() / union)
```

10.4 Why foreground-IoU and not instance-IoU?

Instance-IoU would require **matching** each nucleus in method A to a nucleus in method B, then computing IoU of matched pairs. Matching is itself ambiguous (a nucleus can be over-segmented in one method into two pieces) and typically requires solving a Hungarian assignment problem. That's a one-extra-pass-of-work for marginal extra information.

Foreground-IoU answers a slightly weaker question: *"do the methods agree on **where** nuclei are, regardless of how many they split into?"* For our purpose (validating without GT) that's exactly what we need.

10.5 Our headline numbers

Cohort	classical vs cellpose	classical vs stardist	cellpose vs stardist
BRC	0.57	0.63	0.62
CRC	0.55	0.57	0.62
HCC	0.66	0.64	0.67
NSCLC_AD	0.51	0.52	0.66
NSCLC_SCC	0.56	0.57	0.60

Reading the table: every entry is between 0.51 and 0.67. For foreground masks generated by *independently engineered* systems on *un-annotated* data, anything above 0.50 is strong agreement. (For context, supervised models with full GT typically score 0.70–0.90 on similar datasets.)

10.6 Per-cohort intuition

- **HCC has the highest IoU** (~0.65 across pairs). Hepatocyte nuclei are large, sparse, and well-separated → easy targets where all three methods converge.
- **NSCLC_AD has slightly lower classical-vs-DL IoU** (~0.51). Glandular patterns produce cells with ambiguous boundaries; the watershed prior fights with the polygon prior here.
- **CRC scatter is the widest** (highest count variance), but mean IoU is still 0.55+ — the methods agree on **most** images, just disagree noticeably on a few dense lymphocyte regions.

10.7 What strong agreement does *not* prove

- It does **not** prove all three methods are correct.
- All three could share a systematic blind spot (e.g. all under-detecting very small lymphocytes the same way).
- Foreground-IoU does not check that nuclei are *correctly partitioned* into instances — only that the foreground regions overlap.

This is why the "Limitations" section of the report is honest about needing eventual ground-truth validation.

10.8 Code reference

- `code/02_compare_methods.py` builds `results/method_agreement.csv` and the count scatter plot.

10.9 Why IoU $>$ 0.5 is genuinely strong here

A reviewer might say "0.6 IoU sounds low — supervised models hit 0.85+". That's true *with* labels. We are doing something different:

- We compare two **un-supervised** segmentations against **each other**.
- Each method has its own definition of where a nucleus boundary is (watershed line, flow attractor basin, polygon edge). Even if both are "correct", their boundaries can disagree by 2–3 pixels per nucleus.
- That 2–3 pixel disagreement compounds across thousands of nuclei to pull mean IoU down by ~0.20–0.30, even when both methods correctly localise every nucleus.

So a foreground IoU of 0.55 between Cellpose and StarDist is roughly equivalent to **"both methods detected the same nuclei but drew slightly different boundaries"**. That is the strongest validation we can produce without ground truth.

10.10 What would change with ground truth

With manual annotations we could compute, *per method*:

Metric	Definition
Detection precision	true positives / predicted nuclei
Detection recall	true positives / annotated nuclei
F1	harmonic mean of precision and recall
Panoptic Quality (PQ)	F1 $\times$ mean(IoU of matched pairs)
Aggregated Jaccard (AJI)	Hungarian-matched nucleus IoU

These are the standard metrics for nucleus segmentation papers (e.g. the MoNuSeg challenge). They require ~5 minutes of annotation per crop and would be the first thing to do if extending this work.

10.11 Building the method-agreement CSV — what each row means

```
# results/method_agreement.csv columns:
# cohort, image, classical_count, cellpose_count, stardist_count,
# iou_classical_cellpose, iou_classical_stardist, iou_cellpose_stardist
```

Reading one row tells you: "on this specific image, classical and Cellpose detected close counts, with foreground masks overlapping at IoU 0.61". If a row has IoU $<$ 0.40 it is worth looking at the image manually — it is almost always a particularly dense lymphocyte cluster or an artefact.

10.12 Aggregation choice: mean vs median

We report **mean** per cohort. The distribution of per-image IoU is fairly symmetric, so mean and median agree to two decimals. We chose mean because it is the more common reporting convention and because standard error is straightforward to compute on top of it.

Chapter 11

10 — Results interpretation by cohort

This file connects the numbers in `results/per_cohort_summary.csv` to real histology. Read it together with the qualitative grid at `results/figures/01_qualitative_grid_classical.png`.

11.1 Reference table (classical pipeline, mean per image)

Cohort	Count	Density / Mpx	Mean area (px²)	Eccentricity	Notes
BRC	570	2,048	117	0.66	High density, large variance
CRC	539	1,935	130	0.70	Largest nuclei, glands + lympho-cytes
HCC	356	**1,279**	118	0.72	**Lowest density**
NSCLC_AD	460	1,652	96	0.71	Moderate, glandu-lar
NSCLC_SCC	499	1,792	**80**	0.67	**Smallest, most uniform**

11.2 BRC — Breast carcinoma

- **High density (~2,048)** with large standard deviation across images.
- Mean area mid-range (~117 px²).
- Eccentricity ~0.66 — moderate, neither very round nor very elongated.

Why: breast carcinoma tissue typically contains a mix of: - Tumour glands (epithelial cells with medium nuclei). - Stroma (fewer, sparser nuclei). - Lymphocytic infiltrate (small, dense, round nuclei).

The mix produces high density on average and high variance image-to-image.

11.3 CRC — Colorectal carcinoma

- High density (~1,935).
- **Largest mean nucleus area** (~130 px²) — pulled up by big columnar epithelial nuclei lining gland crypts.
- Highest eccentricity (~0.70) — columnar epithelial cells have cigar-shaped nuclei.

Why: CRC tissue is dominated by glandular crypts surrounded by stroma. The columnar epithelial cells have characteristic elongated, larger nuclei.

11.4 HCC — Hepatocellular carcinoma

- **Lowest density (~1,279)** by a wide margin.
- Mean area mid-range (~118 px²).
- Highest eccentricity (~0.72), but with low std → nuclei are uniformly somewhat elongated.

Why: hepatocytes are large cells with **abundant cytoplasm**. The nuclei themselves are not unusually small — there are just **fewer nuclei per unit area** because each cell takes up more space. This is the most distinctive pattern in our 5 cohorts and all three methods agree on it.

11.5 NSCLC_AD — Non-small-cell lung carcinoma, adenocarcinoma

- Moderate density (~1,652).
- **Smallest mean area among non-SCC cohorts** (~96 px²).
- Eccentricity ~0.71.

Why: adenocarcinoma forms glandular structures with cuboidal/ columnar cells. The nuclei are smaller than in CRC (different epithelium) but still arranged in regular gland-lining patterns.

11.6 NSCLC_SCC — Non-small-cell lung carcinoma, squamous cell carcinoma

- Above-average density (~1,792).
- **Smallest mean nucleus area (~80 px²)**.
- **Tightest area distribution** (std ~9 px² vs 22–67 in other cohorts) → highly *uniform* nucleus size.
- Eccentricity ~0.67 — moderate.

Why: SCC forms **dense, monomorphic squamous cell nests**. Cells are packed tightly, nuclei are small and remarkably uniform in size. All three methods independently rank NSCLC_SCC as the cohort with the smallest, most uniform nuclei.

11.7 What the patterns tell a reviewer

When asked **"do your numbers make biological sense?"** the answer is yes, in three independent ways:

1. **Density ordering** matches expectation: HCC < NSCLC_AD < NSCLC_SCC ≈ CRC < BRC.
2. **Size ordering** matches expectation: NSCLC_SCC has the smallest nuclei, CRC the largest.
3. **All three methods agree on the rank order** of cohorts on every metric (density, mean area). This consistency is itself evidence that the segmentation is biologically meaningful.

11.8 Caveats

- **No ground truth** → we cannot say "BRC truly has 5,704 nuclei".
- **Counts are sensitive to image size** — that's why density is the preferred cohort-level metric.
- **Per-image variance is large** in BRC and CRC because tissue composition varies dramatically (a gland-dense crop is very different from a stroma-dense crop).

11.9 Code reference

- Per-cohort CSV: `results/per_cohort_summary.csv`

- Box plots: `results/figures/02_count_by_cohort.png`, `03_area_by_cohort.png`
- Density bar: `results/figures/04_density_by_cohort.png`

11.10 Sanity checks a pathologist would run

Before trusting these numbers, a pathologist would ask:

1. **"Is HCC really the lowest density?"** Yes — hepatocytes are large and there are far fewer per area unit. Other published H&E pipelines on HCC report similar density ranges.
2. **"Are NSCLC_SCC nuclei really uniform?"** Yes — SCC by definition forms monomorphic squamous nests. Std of nucleus area being ~9 px² (vs 39–67 in other cohorts) is the quantitative signature.
3. **"Why is BRC variance so high?"** Because BRC tissue can range from nearly all stroma (sparse) to nearly all tumour glands + lymphocytic infiltrate (dense). 10 random crops capture this diversity.
4. **"Mean solidity ~0.93 — are those really nuclei?"** Real nuclei *are* ~0.92–0.96 solid. If you got 0.7 you'd worry about jagged false positives; if you got 0.99 you'd worry about overly-smoothed regions that have lost real boundary detail.

All four questions resolve in our favour, which is why we are confident in the headline numbers despite no ground truth.

11.11 What we would do differently with more time

- Plot **density vs nucleus area** as a 2-D scatter — cohorts would separate cleanly into clusters (a proxy for cohort classification *from morphometrics alone*).
- Compute **inter-image variance** within cohort and compare to inter-cohort variance — quantifies how distinguishable cohorts really are.
- Compute **size distribution percentiles** (P10, P50, P90) per cohort — more informative than mean + std for a long-tailed distribution.

Chapter 12

11 — Codebase walkthrough

A guided tour of every file in `code/`. Each entry says *what the file is for*, *what to read in it*, and *what depends on it*.

12.1 Directory map

```
code/
|-- ../requirements.txt              ← classical pipeline deps (at repo root)
|-- ../requirements_deeplearning.txt ← extra deps for Cellpose / StarDist (at repo root)
|
|-- config.py                     ← paths, cohort list, classical params
|-- image_io.py                   ← load images, save masks/overlays
|-- features.py                   ← per-image morphometric stats
|-- visualisation.py              ← overlay drawing, qualitative grids
|
|-- method_classical.py           ← classical segmenter (no DL)
|-- method_cellpose.py            ← Cellpose v4 wrapper
|-- method_stardist.py            ← StarDist wrapper (local model load)
|
|-- 01_run_segmentation.py        ← driver: run ONE method on all 50 images
|-- 02_compare_methods.py         ← cross-method IoU + scatter figure
|-- 03_make_report_figures.py     ← per-cohort figures + summary CSV
|-- make_pptx.py                  ← build presentation/slides.pptx
```

12.2 The four "library" files

These contain reusable helpers and zero CLI logic.

12.2.1 `config.py`

Single source of truth for: - `INPUT_DIR`, `RESULTS_DIR`, `FIGURES_DIR` — path constants. - `COHORTS` — list of cohort folder names. - `CLASSICAL` — a frozen `ClassicalParams` dataclass with all the classical-pipeline thresholds.

If you want to retune, edit ONE place. Everything else imports from here.

12.2.2 `image_io.py`

Three small functions: - `load_rgb(path)` — read PNG to HxWx3 uint8. - `save_mask(labels, path)` — write uint16 PNG (instances). - `save_overlay(rgb_overlay, path)` — write the overlay PNG. - `gather_inputs(input_dir, cohorts)` — return a sorted `[(cohort, path), ...]` list for the entire dataset. - `output_paths(method, cohort, image_stem, results_dir)` — compute the canonical overlay/mask file paths.

12.2.3 features.py

One function: - `extract_features(labels, image_shape)` → dict with all the columns documented in 09_morphometrics.md.

12.2.4 visualisation.py

Two functions: - `make_overlay(rgb, labels)` — yellow nucleus boundaries + count box. - `qualitative_grid(rows, out_path)` — generic 5×4 figure builder.

12.3 The three "method" files

Each implements the contract `segment(rgb) -> uint32 instance label image` and exposes a `NAME` constant.

12.3.1 method_classical.py

Self-contained classical pipeline. Plus `hematoxylin_channel(rgb)` is exposed because the qualitative grid wants to display the H channel side-by-side with the original.

→ Walked through line-by-line in 02_classical_pipeline.md and 05_watershed_instances.md.

12.3.2 method_cellpose.py

Lazy-loads `models.CellposeModel(gpu=...)`, choosing MPS / CUDA / CPU in that order. Then calls `model.eval(rgb, diameter=None, ...)`.

→ Explained in 06_cellpose.md.

12.3.3 method_stardist.py

Loads from a local copy under `~/.stardist/models/2D_versatile_he/` (bypassing keras' SSL-checking downloader). Calls `predict_instances(img, prob_thresh=0.40, nms_thresh=0.30)`.

→ Explained in 07_stardist.md.

12.4 The three driver scripts (numbered)

12.4.1 01_run_segmentation.py

Runs ONE method on all 50 images.

```
python code/01_run_segmentation.py --method classical
python code/01_run_segmentation.py --method cellpose
python code/01_run_segmentation.py --method stardist
```

Flags: - `--cohort BRC` — restrict to one cohort. - `--limit 1` — process only the first N images (smoke-test).

Implementation: 1. Lazy-import the chosen method module (no TF/Torch cost when only classical is requested). 2. Loop with `tqdm`. 3. Per image: `load_rgb` → `method.segment` → save mask + overlay → `extract_features`. 4. Write `results/<method>/per_image_stats.csv`. 5. Print a per-cohort summary to stdout.

12.4.2 `02_compare_methods.py`

After all three methods are run: - Builds `results/per_image_all_methods.csv` (long-form, one row per (image, method)). - Computes pairwise foreground-mask IoU per image → `results/method_agreement.csv`. - Generates `figures/05_method_count_scatter.png` and `figures/06_method_qualitative_grid.png`.

→ Explained in 08_method_comparison.md.

12.4.3 `03_make_report_figures.py`

Per-cohort aggregate figures + the classical qualitative grid: - `figures/01_qualitative_grid_classical.png` - `figures/02_count_by_cohort.png` - `figures/03_area_by_cohort.png` - `figures/04_density_by_cohort.png` - `results/per_cohort_summary.csv` (3 methods × 5 cohorts = 15 rows)

12.4.4 `make_pptx.py`

Reads the figures + tables and writes `presentation/slides.pptx`. Uses `python-pptx`. 17 slides, 16:9.

12.5 Adding a new method

The architecture makes this trivial. Steps:

1. Create `code/method_yournewname.py` with:

```python
NAME = "yournewname"

def segment(rgb: np.ndarray) -> np.ndarray:
    # ... do whatever, return uint32 instance label image
```

2. Register it in `01_run_segmentation.py`'s `AVAILABLE_METHODS` dict.
3. Run `python code/01_run_segmentation.py --method yournewname`.
4. Re-run `02_compare_methods.py` and `03_make_report_figures.py`.

No other file needs to change.

12.6 What lives outside code/

- `results/` — every artefact (overlays, masks, CSVs, figures).
- `report/report.md` — the long-form write-up.
- `presentation/slides.md` + `slides.pptx` — the deck.
- `study/` — this folder.

12.7 Conventions worth knowing

- **Filenames carry context**: `BRC_Image1_overlay.png` makes sense even if copied out of the tree.
- **Numbered scripts** (`01_…`, `02_…`, `03_…`) advertise the run order.
- **Lazy imports** in `01_run_segmentation.py` mean classical-only runs don't import TF/Torch.
- **One global parameter set** in `CLASSICAL`; never per-cohort tuning.

12.8 Reading a script: a worked walk-through of `01_run_segmentation.py`

```python
# 1. Parse CLI args (--method, --cohort, --limit)
args = parser.parse_args()

# 2. Lazy-import the chosen method module
if args.method == "classical":
    from method_classical import segment, NAME
elif args.method == "cellpose":
    from method_cellpose import segment, NAME
elif args.method == "stardist":
    from method_stardist import segment, NAME

# 3. Build the input list
targets = gather_inputs(INPUT_DIR, COHORTS)
if args.cohort:
    targets = [t for t in targets if t[0] == args.cohort]
if args.limit:
    targets = targets[:args.limit]

# 4. Loop with a progress bar
rows = []
for cohort, path in tqdm(targets, desc=NAME):
    rgb     = load_rgb(path)
    labels  = segment(rgb)            # <-- method-specific
    overlay = make_overlay(rgb, labels)

    op, mp  = output_paths(NAME, cohort, path.stem, RESULTS_DIR)
    save_overlay(overlay, op)
    save_mask(labels, mp)

    feats   = extract_features(labels, rgb.shape)
    feats.update(cohort=cohort, image=path.name)
    rows.append(feats)

# 5. Write the per-image CSV
pd.DataFrame(rows).to_csv(
    RESULTS_DIR / NAME / "per_image_stats.csv", index=False)
```

Five steps. Anything in `code/01_run_segmentation.py` that isn't one of these is just argument parsing or a per-cohort summary print at the end.

12.9 What to do when something breaks

Symptom	Fix
ImportError on `cellpose` / `stardist`	Run `pip install -r requirements_deeplearning.txt`
`cpsam` weights download fails	Check connectivity; rerun `./run.sh weights`
StarDist SSL error	Manually place weights in `~/.stardist/models/2D_versatile_he/`
Cellpose extremely slow on Mac	Verify MPS: `python -c "import torch; print(torch.backends.mps.is_available())"` should print `True`
`KeyError: 'overlay'`	Likely an old run with pre-refactor paths — delete `results/` and re-run

12.10 Adding new morphometrics

If you wanted to add `mean_perimeter` to every CSV:

1. In `code/features.py`, add `"mean_perimeter": float(np.mean([r.perimeter for r in regions]))` to the dict returned by `extract_features`.
2. Re-run all three methods (classical takes ~18s; the DL methods need their CSVs regenerated only if you also re-run them).

Everything downstream (`02_compare_methods.py`, `03_make_report_figures.py`, `make_pptx.py`) reads CSVs by column name and gracefully ignores unknown columns, so nothing else needs changing.

Chapter 13

12 — Likely review questions & answers

A self-quiz / Q&A bank. Read this before any meeting where you have to defend the work.

13.0.1 Q1. *"What's the goal in one sentence?"*

Detect every nucleus in 50 H&E images across 5 cancer cohorts and quantify them, using three independent methods because no ground truth is available.

13.0.2 Q2. *"Why three methods? Wasn't one enough?"*

Without ground truth we have no way to compute precision / recall. The strongest available substitute is **agreement** between methods of **different families**: classical image processing, generalist DL (Cellpose), and H&E-specific DL (StarDist). When all three agree on a number, that number is almost certainly correct. When they disagree, the disagreement is itself diagnostic.

13.0.3 Q3. *"Which method is best?"*

Depends on the criterion. - **Best total-count agreement with the others**: classical and Cellpose are within 0.6%. - **Cleanest boundaries on dense regions**: Cellpose. - **Fastest**: classical (~18 s for 50 images on CPU). - **Most explainable**: classical (every step is a documented image-processing operator).

For a production pipeline I would use Cellpose with classical as a sanity-check fallback.

13.0.4 Q4. *"Walk me through the classical pipeline."*

RGB $\rightarrow$ HED stain deconvolution (keep H channel) $\rightarrow$ percentile clip + Gaussian smooth $\rightarrow$ Otsu threshold $\rightarrow$ opening + hole-fill + drop tiny objects $\rightarrow$ distance-transform watershed (splits touching nuclei) $\rightarrow$ filter by area [30, 4000] px^2 and solidity $\geq$ 0.70. The watershed is the key step — it converts a binary mask into per- instance labels.

Detail in 02_classical_pipeline.md.

13.0.5 Q5. *"Why HED deconvolution and not just the blue channel?"*

The blue channel of an RGB image picks up *everything* that scatters blue light, including pale eosin areas and clean glass. HED deconvolution uses the Beer-Lambert model to invert the known stain matrix and isolate **only the hematoxylin contribution** at each pixel. The result is a clean nucleus-specific channel.

Detail in 03_stain_deconvolution.md.

13.0.6 Q6. *"What is the watershed step actually doing?"*

After thresholding, two touching nuclei share one binary blob. The distance transform turns each nucleus into a "mountain" with its peak at the centre. We find local peaks (`peak_local_max`, min distance 6 px) and use them as markers for `watershed`, which floods outward from each peak. Where two flood fronts meet, that's the boundary between the two nuclei.

Detail in 05_watershed_instances.md.

13.0.7 Q7. *"What does Cellpose do internally?"*

Cellpose v4 (`cpsam`) is a SAM-based transformer that, for every pixel, predicts a 2D flow vector pointing to the centre of its nucleus, plus a cell-probability. Instances come from gradient ascent on the flow field — pixels that converge to the same point are the same nucleus. No explicit watershed needed.

Detail in 06_cellpose.md.

13.0.8 Q8. *"And StarDist?"*

Predicts a star-convex polygon for every pixel: 32 ray distances to the boundary plus a centre-probability. NMS picks the best non- overlapping polygons.

The polygon prior is a strong shape constraint — great for typical ellipse-shaped nuclei, conservative on overlapping or non-convex ones.

Detail in 07_stardist.md.

13.0.9 Q9. *"Why did you tune StarDist's `prob_thresh` from 0.69 to 0.40?"*

The default missed roughly half the nuclei in dense lymphocyte clusters in BRC and CRC (smoke-test on `BRC/Image1.png`: 216 vs ~580 from the other methods). Lowering to 0.40 produced 495 — much closer to classical (577) and Cellpose (590), with no qualitative over-segmentation introduced. 0.40 is a commonly-used value for `2D_versatile_he` on dense H&E.

13.0.10 Q10. *"How can you trust your numbers without ground truth?"*

Three lines of evidence: 1. **Cross-method count agreement**: classical 24,250, Cellpose 24,097 (0.6% gap), StarDist 20,783 (–14%, expected for the conservative polygon prior). 2. **Foreground-mask IoU between methods**: 0.51–0.67 across cohorts. Strong for un-annotated data. 3. **Biological plausibility**: density ranking HCC < NSCLC_AD < NSCLC_SCC ≈ CRC < BRC matches the known histology of these tumour types.

Detail in 08_method_comparison.md and 10_results_interpretation.md.

13.0.11 Q11. *"What are the limitations of your work?"*

1. No ground truth → no precision/recall.
2. Watershed in the classical pipeline can over-segment large nuclei.
3. StarDist polygon prior misses overlapping/non-convex nuclei.
4. Stain variability remains; Macenko normalisation would help.
5. Single global parameter set across cohorts (chosen for fairness, not maximum per-cohort accuracy).

13.0.12 Q12. *"How would you improve this in production?"*

1. **Manual annotation** of ~5 crops per cohort for real F1 / PQ.
2. **Macenko / Reinhard stain normalisation** before all methods.
3. **HoVer-Net** for joint segmentation + nucleus-type classification (epithelial / lymphocyte / stromal counts per cohort).
4. **Method ensemble**: union-of-masks with NMS to combine the three methods into a single consensus segmentation.
5. **Hungarian-matched instance IoU** for proper per-nucleus correspondence between methods.

13.0.13 Q13. *"Why is HCC the lowest density?"*

Hepatocellular carcinoma tissue is dominated by hepatocytes — large cells with abundant pink (eosin) cytoplasm. The nuclei themselves are not unusually small; there are just **fewer cells per unit tissue area**. All three of our methods independently rank HCC as the lowest-density cohort, so this is not a method artefact.

13.0.14 Q14. *"Why does NSCLC_SCC have the smallest, most uniform nuclei?"*

Squamous cell carcinoma forms dense monomorphic nests of cells with small, tightly-packed, similarly-sized nuclei. The standard deviation of nucleus area in NSCLC_SCC (~9 px²) is much smaller than in NSCLC_AD (~22 px²) or CRC (~67 px²), confirming this uniformity quantitatively.

13.0.15 Q15. *"How fast is the pipeline?"*

Step	Time on MacBook Pro (Apple Silicon)
Classical (50 images, CPU)	~18 s
StarDist (50 images, CPU)	~30 s
Cellpose (50 images, **MPS**)	~5 min
Cellpose (50 images, CPU only)	~2 h (would not be practical)
Comparison + figures + pptx	~10 s total

Apple's MPS backend gives Cellpose a ~25× speedup over CPU. No NVIDIA GPU required.

13.0.16 Q16. *"What if I asked you to do this on 5,000 images instead of 50?"*

Steps: 1. Move Cellpose to a GPU machine (would be ~50× faster again on a single H100). 2. Parallelise the classical pipeline with `joblib` or `concurrent.futures` — embarrassingly parallel per image. 3. Stream image loading; don't keep them all in memory. 4. Persist masks as compressed PNGs (already done) or zarr/HDF5 if going larger. 5. Likely move to a tile-based workflow if images become whole-slide (gigapixel) instead of small ROIs.

13.0.17 Q17. *"What's `mean_solidity` and why ~0.93?"*

Solidity = `region_area / convex_hull_area`. A perfectly convex shape has solidity 1. Real nuclei are nearly convex (small bumps and indentations), giving solidity ~0.92–0.96. The fact that all five cohorts average ~0.93 is a *positive sign* — our segmented instances look like real nuclei, not jagged fragments.

Definitions in 09_morphometrics.md.

13.0.18 Q18. *"Could two methods agree and still both be wrong?"*

Yes — that's why agreement is *evidence*, not *proof*. If all three methods share a systematic blind spot (e.g. all under-detecting faintly-stained nuclei) they can agree on a wrong answer. The ultimate fix is manual annotation of a held-out subset, which is listed as a top improvement in the report.

13.0.19 Q19. *"What would change if I gave you ground truth right now?"*

I would: 1. Run all three methods on the annotated subset. 2. Compute precision, recall, F1, and panoptic quality per method. 3. Pick the best method overall as the "primary". 4. Use the other two as automatic sanity-check baselines that flag images where they disagree with the primary by more than a cohort-specific threshold (those go to a human reviewer).

13.0.20 Q20. *"Where is everything?"*

```
task/
|-- code/                ← Python (4 modules + 4 scripts)
|-- doc/Assessment/      ← input PNGs (provided)
|-- results/
|   |-- classical/       ← 50 overlays, 50 masks, per_image_stats.csv
|   |-- cellpose/        ← (same)
|   |-- stardist/        ← (same)
|   |-- per_cohort_summary.csv
|   |-- method_agreement.csv
|   |-- figures/         ← 6 figures used in the report
|-- report/report.md
|-- presentation/
|   |-- slides.md
|   |-- slides.pptx
|-- study/               ← this folder
```

End of Q&A. If you can answer 15+ of these confidently, you can defend the project in any interview/review setting.

13.0.21 Q21. *"How would you debug a single image where StarDist disagrees badly with the others?"*

1. Open `results/method_agreement.csv`, sort by `iou_classical_stardist`, pick the lowest-IoU row.
2. Open the three overlay PNGs side-by-side (`results/<method>/overlays/<COHORT>_<image>_overlay.png`).
3. Visually inspect: where are nuclei detected by classical/Cellpose but missed by StarDist? Are they in dense clusters, near the image edge, or unusually shaped?
4. Lower `prob_thresh` further for that image (e.g. 0.30) and re-run just that image with `--cohort X --limit 1` to confirm whether they reappear.
5. If they do, the problem is StarDist's confidence calibration on that tissue type — reportable as a known limitation.

13.0.22 Q22. *"Why is your classical pipeline a single Python file rather than split into modules?"*

The classical pipeline IS one logical operation: "H&E image → instance mask". Splitting it into one-function-per-step modules would add import overhead without any clarity benefit — the file is only ~80 lines. The operations that ARE split out (image I/O, features, visualisation, config) are reused across all three methods.

13.0.23 Q23. *"What's the difference between watershed and the polygon prior in StarDist?"*

Both are answers to "how do I split touching nuclei".

- **Watershed** is *post hoc*: detect a binary blob first, then split. Splits are based on the geometry of the blob (distance peaks).
- **Polygon prior** is *built in*: each nucleus is a polygon defined by its centre, so two centres = two polygons.

Watershed is more flexible (any blob shape) but can over-segment. Polygon prior is more constrained (must be star-convex) but cannot over-segment by construction.

13.0.24 Q24. *"How robust is your pipeline to image resolution / magnification?"*

- **Classical**: parameters (min_area, opening disk radius, watershed min_distance) are in absolute pixels, so they assume a particular magnification. For different magnification we'd scale them proportionally.
- **Cellpose**: `diameter=None` auto-estimates per image — robust to moderate magnification changes.

- **StarDist**: assumes magnification roughly matches its training data; for very different mag we'd resize the image first.

All three of our cohorts use comparable magnification, so this wasn't a problem here.

13.0.25 Q25. *"Could you run this on a whole-slide image (gigapixel)?"*

Not directly — our images fit in memory. For a WSI we would: 1. Tile the slide into 1024-px patches with ~64 px overlap. 2. Run any of the three methods per tile. 3. Stitch instance masks back, resolving boundary nuclei via IoU matching across tile borders. 4. Aggregate counts and morphometrics at the slide level.

Cellpose has a built-in tiling helper. The classical pipeline would need a custom tiling layer (~50 lines of code).